

Operating Systems Lab

Complete Visual Guide — Environment Setup, Program 1 (fork) & Program 2 (FCFS / SJF)

Platform: Ubuntu 24.04 on WSL (Windows) | VS Code | Python 3.12.3 (.venv)

This guide walks through every captured screenshot in the order the work was actually done — setting up the lab environment, running the first fork program, and building, running and archiving Program 2 (FCFS and non-preemptive SJF scheduling). Concepts, code walkthroughs, expected outputs, common mistakes and a final checklist are included so the document works as a standalone lab record.

1. Lab Environment Setup

The lab lives in **~/Operating-Systems-Lab** on Ubuntu (WSL), inside an isolated Python virtual environment (**.venv**) so the lab packages never touch the system Python.

1.1 Installing the Python stack

```
tusharkapoor@DESKTOP-0VB X + v
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: six, pyparsing, psutil, pillow, packaging, numpy, kiwisolver, fonttools, cyclr, python-dateutil, contourpy, pandas, matplotlib
Successfully installed contourpy-1.3.3 cyclr-0.12.1 fonttools-4.64.0 kiwisolver-1.5.1 matplotlib-3.11.1 numpy-2.5.2 packaging-26.3 pandas-3.0.5 pillow-12.3
0 psutil-7.2.2 pyparsing-3.3.2 python-dateutil-2.9.0.post0 six-1.17.0
(.venv) tusharkapoor@DESKTOP-0VB17SS:~/Operating-Systems-Lab$ python -c "import psutil, pandas, matplotlib; print('PYTHON LAB ENVIRONMENT READY!')"
PYTHON LAB ENVIRONMENT READY
(.venv) tusharkapoor@DESKTOP-0VB17SS:~/Operating-Systems-Lab$ touch README.md requirements.txt .gitignore
python -m pip freeze > requirements.txt

# Create common folders and all Lab1-Lab4, Capstone directories
mkdir -p common/{templates,test-data,screenshots,reports}
mkdir -p Lab1/Q1_System_Calls/{src,input,output,screenshots,tests}
mkdir -p Lab1/Q2_FCFS_SJF/{src,input,output,screenshots,tests}
mkdir -p Lab1/Q3_Priority_Round_Robin/{src,input,output,screenshots,tests}
mkdir -p Lab1/Q4_Metrics_Threads_IPC/{src,input,output,screenshots,tests}
mkdir -p Lab2/Q1_Thread_Models/{src,input,output,screenshots,tests}
mkdir -p Lab2/Q2_Race_Mutex_Semaphore/{src,input,output,screenshots,tests}
mkdir -p Lab2/Q3_Classical_Synchronization/{src,input,output,screenshots,tests}
mkdir -p Lab2/Q4_Deadlock_Banker/{src,input,output,screenshots,tests}
mkdir -p Lab3/Q1_Contiguous_Swapping/{src,input,output,screenshots,tests}
mkdir -p Lab3/Q2_Paging_Segmentation/{src,input,output,screenshots,tests}
mkdir -p Lab3/Q3_Page_Replacement/{src,input,output,screenshots,tests}
mkdir -p Lab3/Q4_Device_Management/{src,input,output,screenshots,tests}
mkdir -p Lab3/Q5_Disk_Scheduling/{src,input,output,screenshots,tests}
mkdir -p Lab4/Q1_File_Operations/{src,input,output,screenshots,tests}
mkdir -p Lab4/Q2_Direct_Indexed_Directory/{src,input,output,screenshots,tests}
mkdir -p Lab4/Q3_File_Allocation/{src,input,output,screenshots,tests}
mkdir -p Lab4/Q4_Free_Space_Permissions/{src,input,output,screenshots,tests}
mkdir -p Capstone/{src,data,logs,outputs,screenshots,tests}
(.venv) tusharkapoor@DESKTOP-0VB17SS:~/Operating-Systems-Lab$ touch README.md requirements.txt .gitignore
python -m pip freeze > requirements.txt

# Create common folders and all Lab1-Lab4, Capstone directories
mkdir -p common/{templates,test-data,screenshots,reports}
mkdir -p Lab1/Q1_System_Calls/{src,input,output,screenshots,tests}
mkdir -p Lab1/Q2_FCFS_SJF/{src,input,output,screenshots,tests}
mkdir -p Lab1/Q3_Priority_Round_Robin/{src,input,output,screenshots,tests}
mkdir -p Lab1/Q4_Metrics_Threads_IPC/{src,input,output,screenshots,tests}
mkdir -p Lab2/Q1_Thread_Models/{src,input,output,screenshots,tests}
```

The banner **PYTHON LAB ENVIRONMENT READY** confirms every required library imports cleanly. Immediately after, `requirements.txt` and `.gitignore` are created.

1.2 Directory skeleton and environment verification

```
mkdir -p Lab4/Q4_Free_Space_Permissions/{src,input,output,screenshots,tests}
mkdir -p Capstone/{src,data,logs,outputs,screenshots,tests}
(.venv) tusharkapoor@DESKTOP-0VB17SS:~/Operating-Systems-Lab$ touch Lab1/Q1_System_Calls/README.md Lab1/Q2_FCFS_SJF/README.md Lab1/Q3_Priority_Round_Robin/README.md Lab1/Q4_Metrics_Threads_IPC/README.md Lab2/Q1_Thread_Models/README.md Lab2/Q2_Race_Mutex_Semaphore/README.md Lab2/Q3_Classical_Synchronization/README.md Lab2/Q4_Deadlock_Banker/README.md Lab3/Q1_Contiguous_Swapping/README.md Lab3/Q2_Paging_Segmentation/README.md Lab3/Q3_Page_Replacement/README.md Lab3/Q4_Device_Management/README.md Lab3/Q5_Disk_Scheduling/README.md Lab4/Q1_File_Operations/README.md Lab4/Q2_Direct_Indexed_Directory/README.md Lab4/Q3_File_Allocation/README.md Lab4/Q4_Free_Space_Permissions/README.md Capstone/README.md
(.venv) tusharkapoor@DESKTOP-0VB17SS:~/Operating-Systems-Lab$ pwd
/home/tusharkapoor/Operating-Systems-Lab
which python
python --version
python -c "import psutil, pandas, matplotlib; print('PYTHON LAB ENVIRONMENT READY!')"
python -c "import os,sys,psutil,pandas,matplotlib; print('OS:',os.name); print('Python:',sys.version.split()[0]); print('CPU:',psutil.cpu_count()); print('Memory:',psutil.virtual_memory().total); print('OPERATING SYSTEMS LAB SETUP READY!')"
/home/tusharkapoor/Operating-Systems-Lab
/home/tusharkapoor/Operating-Systems-Lab/.venv/bin/python
Python 3.12.3
PYTHON LAB ENVIRONMENT READY
OS: posix
Python: 3.12.3
CPU: 12
Memory: 8162783232
```

The diagnostics confirm: project root at `/home/tusharkapoor/Operating-Systems-Lab`, the active interpreter is the venv's Python **3.12.3**, OS is **posix** (Ubuntu/WSL), with **12 CPUs** and **~8 GB RAM**. Each question folder follows the convention `{src, input, output, screenshots, tests}`; Capstone uses `{src, data, logs, outputs, screenshots, tests}`.

1.3 The project in VS Code

File Edit Selection View Go Run Terminal Help

Operating-Systems-Lab

Explorer

Operating-Systems-Lab

> .env

> Capstone

> common

> Lab1

> Lab2

> Lab3

> Lab4

> .gitignore

program1.py

README.md

requirements.txt

program1.py

1 import os

2

3 print("Before Fork")

4 print("Current PID:", os.getpid())

5

6 pid = os.fork()

7

8 if pid == 0:

9 print("\nChild Process")

10 print("Child PID :", os.getpid())

11 print("Parent PID:", os.getppid())

12

13 else:

14 os.wait()

15

16 print("\nParent Process")

17 print("Parent PID:", os.getpid())

18 print("Child PID :", pid)

Problems Output Debug Console Terminal Ports

Installing Python via uv... - Task ✓ + - □ ... | ↺ ✕

Executing task: uv python install

Installed Python 3.14.0 in 430ms

+ cpython-3.14.0-windows-x86_64-none (python3.14.exe)

Sessions

Tip: Use /create-agent to scaffold a custom agent for your workflow.

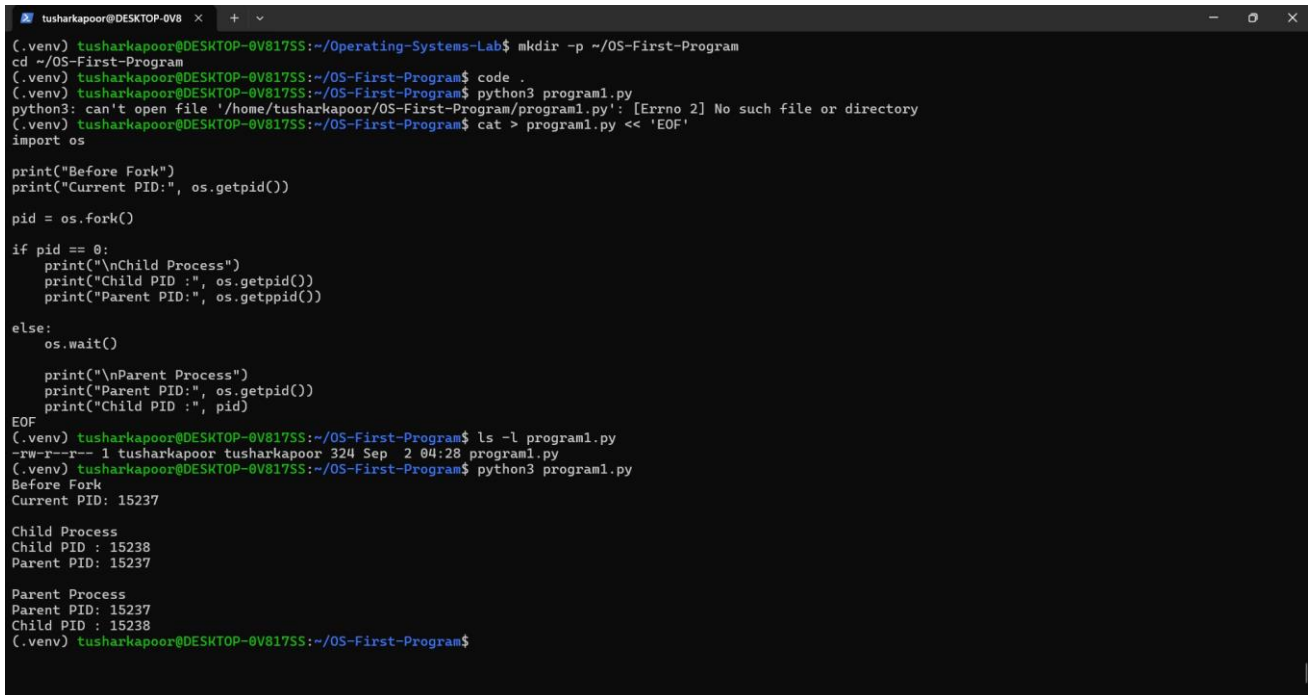
+ program1.py

Describe what to build

+ @ = ↺ ↻ ↶ ↷

2. Program 1 — Process Creation with os.fork()

The first program demonstrates the most fundamental OS call: creating a new process. It was written in the terminal with a heredoc and executed with python3.



```
(.venv) tusharkapoor@DESKTOP-0V817SS:~/Operating-Systems-Lab$ mkdir -p ~/OS-First-Program
(.venv) tusharkapoor@DESKTOP-0V817SS:~/OS-First-Program$ cd ~/OS-First-Program
(.venv) tusharkapoor@DESKTOP-0V817SS:~/OS-First-Program$ code .
(.venv) tusharkapoor@DESKTOP-0V817SS:~/OS-First-Program$ python3 program1.py
python3: can't open file '/home/tusharkapoor/OS-First-Program/program1.py': [Errno 2] No such file or directory
(.venv) tusharkapoor@DESKTOP-0V817SS:~/OS-First-Program$ cat > program1.py << 'EOF'
import os

print("Before Fork")
print("Current PID:", os.getpid())

pid = os.fork()

if pid == 0:
    print("\nChild Process")
    print("Child PID :", os.getpid())
    print("Parent PID:", os.getppid())
else:
    os.wait()

    print("\nParent Process")
    print("Parent PID:", os.getpid())
    print("Child PID :", pid)
EOF
(.venv) tusharkapoor@DESKTOP-0V817SS:~/OS-First-Program$ ls -l program1.py
-rw-r--r-- 1 tusharkapoor tusharkapoor 324 Sep  2 04:28 program1.py
(.venv) tusharkapoor@DESKTOP-0V817SS:~/OS-First-Program$ python3 program1.py
Before Fork
Current PID: 15237

Child Process
Child PID : 15238
Parent PID: 15237

Parent Process
Parent PID: 15237
Child PID : 15238
(.venv) tusharkapoor@DESKTOP-0V817SS:~/OS-First-Program$
```

Figure 2.1 — program1.py executed: parent PID 15237 forks child PID 15238, parent waits, then prints.

Line-by-line behaviour:

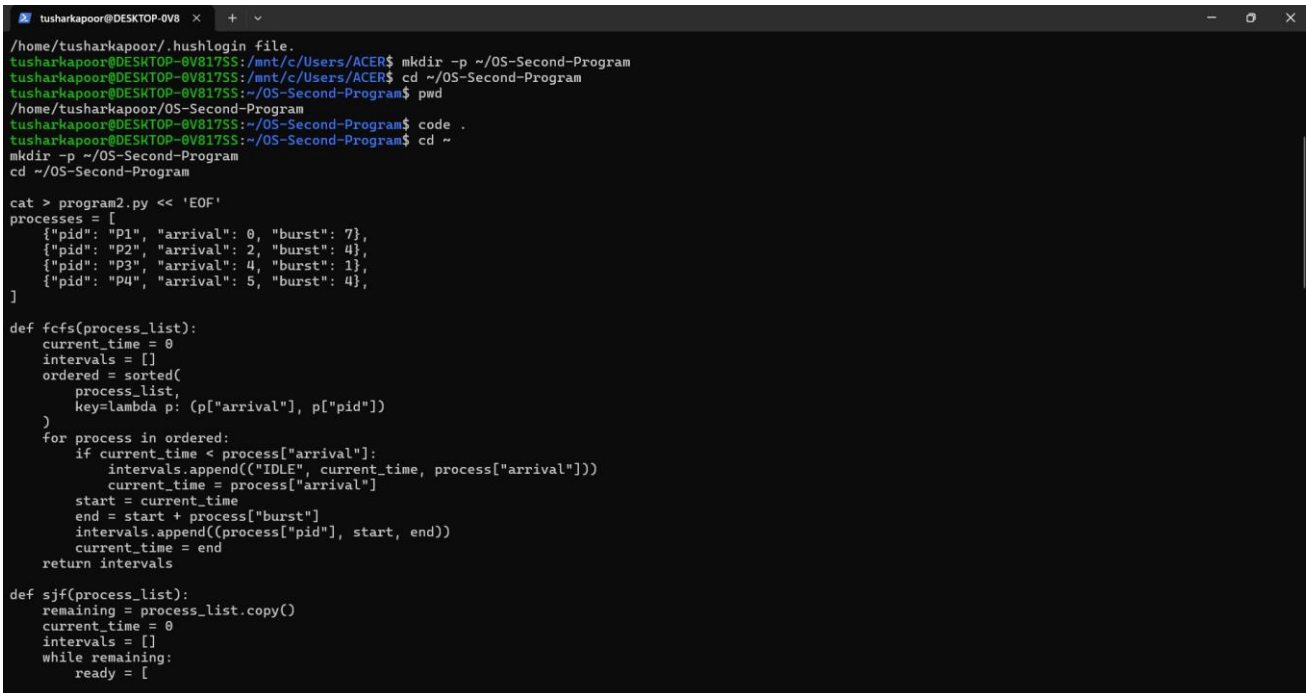
- 1. Before the fork, "Before Fork" and the current PID (**15237**) print once — the process is still single.
- 2. `os.fork()` clones the process: the parent receives the child's PID, the child receives 0.
- 3. Child branch (`pid == 0`): prints Child PID **15238**, Parent PID **15237** — the child's PID is exactly one more, a freshly allocated PID.
- 4. Parent branch: `os.wait()` blocks until the child exits (no zombie), then prints Parent PID 15237 / Child PID 15238.

Child output appearing before parent output confirms the `wait()` worked correctly.

3. Program 2 — FCFS and Non-Preemptive SJF Scheduling

Objective: with the **same four processes**, simulate FCFS (First Come, First Served) and non-preemptive SJF (Shortest Job First), then display the input table, execution intervals and process sequence. Scope is limited to scheduling order and intervals — waiting/turnaround times, Priority and Round Robin come in later programs.

3.1 Create the workspace and open VS Code

A terminal window titled 'tusharkapoor@DESKTOP-0V8' shows the following commands and output:

```
/home/tusharkapoor/.hushlogin file.
tusharkapoor@DESKTOP-0V817SS:/mnt/c/Users/ACER$ mkdir -p ~/OS-Second-Program
tusharkapoor@DESKTOP-0V817SS:/mnt/c/Users/ACER$ cd ~/OS-Second-Program
tusharkapoor@DESKTOP-0V817SS:~/OS-Second-Program$ pwd
/home/tusharkapoor/OS-Second-Program
tusharkapoor@DESKTOP-0V817SS:~/OS-Second-Program$ code .
tusharkapoor@DESKTOP-0V817SS:~/OS-Second-Program$ cd ~
mkdir -p ~/OS-Second-Program
cd ~/OS-Second-Program

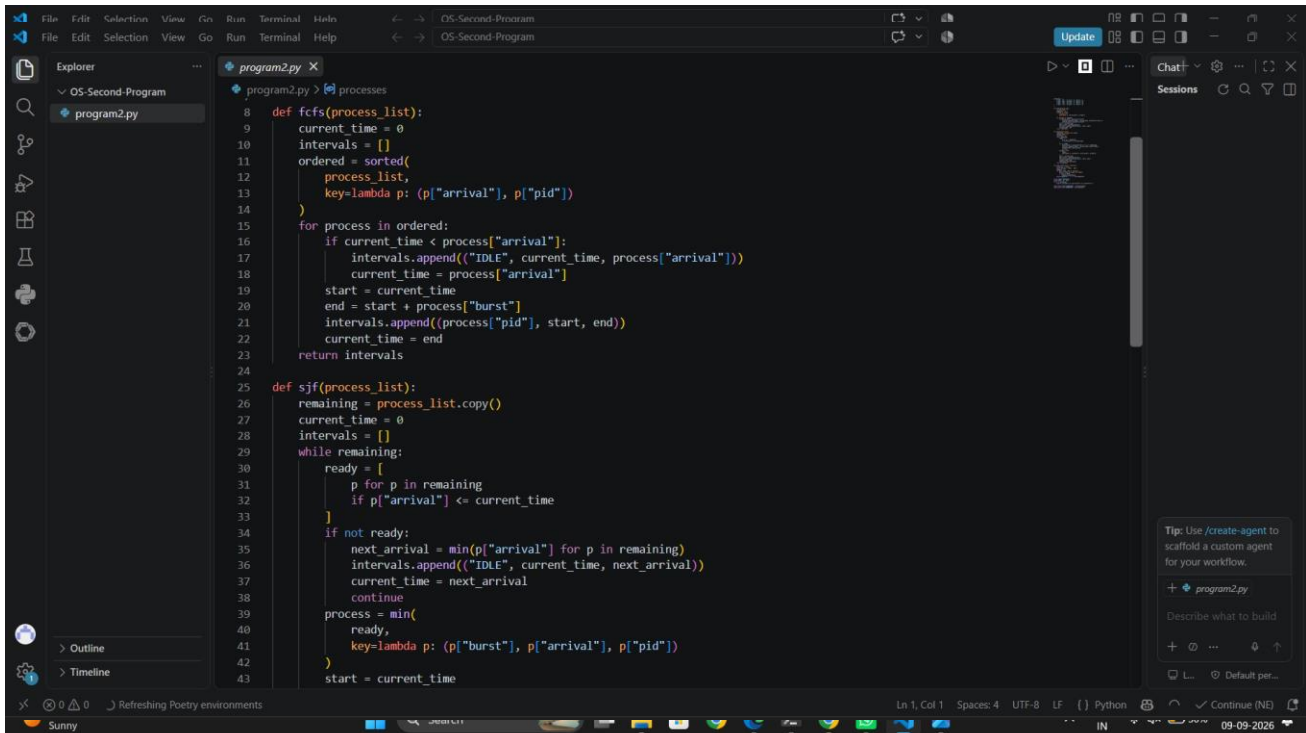
cat > program2.py << 'EOF'
processes = [
    {"pid": "P1", "arrival": 0, "burst": 7},
    {"pid": "P2", "arrival": 2, "burst": 4},
    {"pid": "P3", "arrival": 4, "burst": 1},
    {"pid": "P4", "arrival": 5, "burst": 4},
]

def fcfs(process_list):
    current_time = 0
    intervals = []
    ordered = sorted(
        process_list,
        key=lambda p: (p["arrival"], p["pid"])
    )
    for process in ordered:
        if current_time < process["arrival"]:
            intervals.append(("IDLE", current_time, process["arrival"]))
            current_time = process["arrival"]
        start = current_time
        end = start + process["burst"]
        intervals.append((process["pid"], start, end))
        current_time = end
    return intervals

def sjf(process_list):
    remaining = process_list.copy()
    current_time = 0
    intervals = []
    while remaining:
        ready = [
```

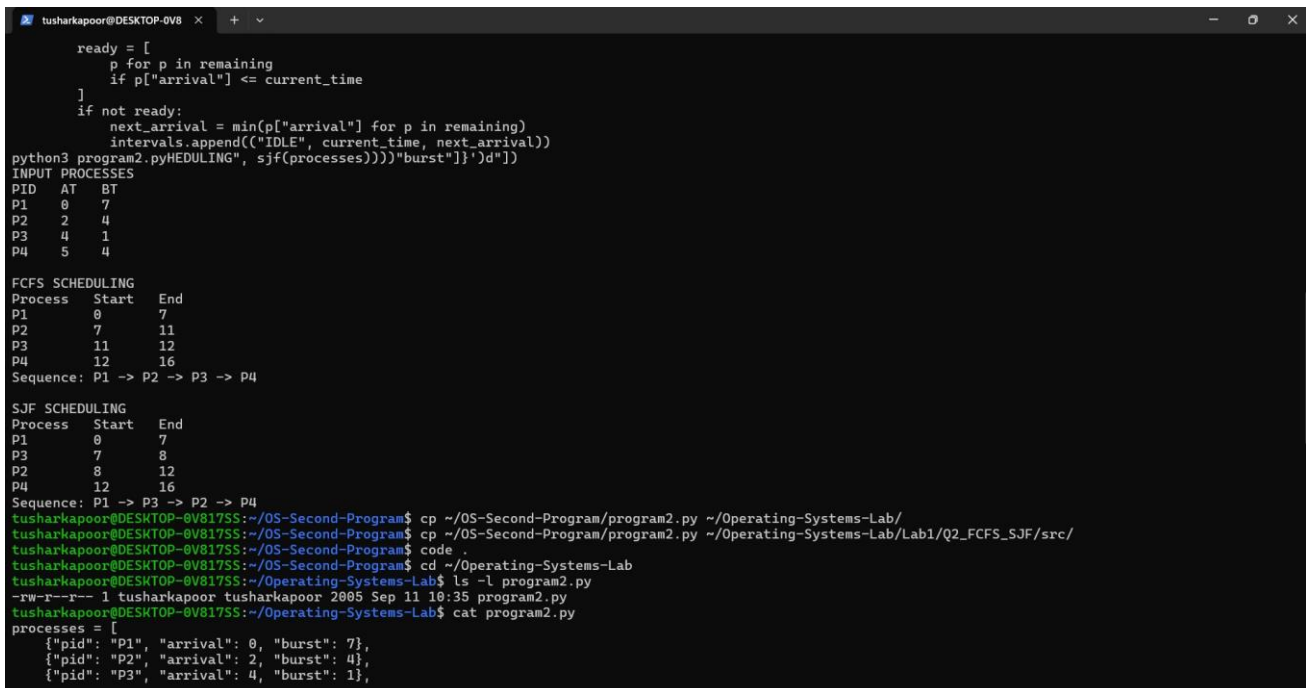
/home/tusharkapoor/OS-Second-Program, and `code .` opens VS Code on that folder. Below that, the `cat > program2.py << 'EOF'` heredoc starts streaming the program into the file straight from the terminal.

3.2 The code in VS Code



```
8 def fcfs(process_list):
9     current_time = 0
10    intervals = []
11    ordered = sorted(
12        process_list,
13        key=lambda p: (p["arrival"], p["pid"])
14    )
15    for process in ordered:
16        if current_time < process["arrival"]:
17            intervals.append(("IDLE", current_time, process["arrival"]))
18            current_time = process["arrival"]
19        start = current_time
20        end = start + process["burst"]
21        intervals.append((process["pid"], start, end))
22        current_time = end
23    return intervals
24
25 def sjf(process_list):
26     remaining = process_list.copy()
27     current_time = 0
28     intervals = []
29     while remaining:
30         ready = [
31             p for p in remaining
32             if p["arrival"] <= current_time
33         ]
34         if not ready:
35             next_arrival = min(p["arrival"] for p in remaining)
36             intervals.append(("IDLE", current_time, next_arrival))
37             current_time = next_arrival
38             continue
39         process = min(
40             ready,
41             key=lambda p: (p["burst"], p["arrival"], p["pid"])
42         )
43         start = current_time
```

3.3 Run the program and check the output



```
tusharkapoor@DESKTOP-0V8:
ready = [
  p for p in remaining
  if p["arrival"] <= current_time
]
if not ready:
  next_arrival = min(p["arrival"] for p in remaining)
  intervals.append(("IDLE", current_time, next_arrival))
python3 program2.pyHEDULING", sjf(processes)))"burst"]}')d"]
INPUT PROCESSES
PID  AT  BT
P1   0   7
P2   2   4
P3   4   1
P4   5   4

FCFS SCHEDULING
Process  Start  End
P1       0    7
P2       7   11
P3      11   12
P4      12   16
Sequence: P1 -> P2 -> P3 -> P4

SJF SCHEDULING
Process  Start  End
P1       0    7
P3       7    8
P2       8   12
P4      12   16
Sequence: P1 -> P3 -> P2 -> P4

tusharkapoor@DESKTOP-0V817SS:~/OS-Second-Program$ cp ~/OS-Second-Program/program2.py ~/Operating-Systems-Lab/
tusharkapoor@DESKTOP-0V817SS:~/OS-Second-Program$ cp ~/OS-Second-Program/program2.py ~/Operating-Systems-Lab/Lab1/Q2_FCFS_SJF/src/
tusharkapoor@DESKTOP-0V817SS:~/OS-Second-Program$ code .
tusharkapoor@DESKTOP-0V817SS:~/Operating-Systems-Lab$ cd ~/Operating-Systems-Lab
tusharkapoor@DESKTOP-0V817SS:~/Operating-Systems-Lab$ ls -l program2.py
-rw-r--r-- 1 tusharkapoor tusharkapoor 2005 Sep 11 10:35 program2.py
tusharkapoor@DESKTOP-0V817SS:~/Operating-Systems-Lab$ cat program2.py
processes = [
  {"pid": "P1", "arrival": 0, "burst": 7},
  {"pid": "P2", "arrival": 2, "burst": 4},
  {"pid": "P3", "arrival": 4, "burst": 1},
```